

KairoX — Material Service API Guide (for Frontend)

1. What this service is

The material service is the **human-driven stock system for the factory floor**. A manager types in a lot when material physically arrives; the system gives that lot a printed barcode and tracks three numbers against it forever.

It is **not** the inventory module (that one is BOM-driven, Phase 2, keyed to a bom_id). Do not call /inventory/* for anything on these screens.

The four things it owns

Concern	What it means
Lots	The material master. One row per <i>material spec</i> , not per delivery.
Stock	on_hand, reserved, available — read-only aggregates.
Receiving	Approved qty adds stock; rejected qty is logged against the supplier.
Supplier orders	A two-state manual PO: ORDERED → ARRIVED.

2. Conventions that apply to every endpoint

Base URL

All routes are mounted under /api/v1.

POST /api/v1/materials/lots

GET /api/v1/materials/stock

POST /api/v1/suppliers/orders

Auth

Every route requires a JWT. Send it as Authorization: Bearer <token>. There is no anonymous material route. Legacy employee-role tokens are rejected at the router level before the handler runs.

Roles — three tiers

The service uses exactly three permission groups. Build your UI gating from these:

STOCK_READERS — can read spec, lots, lot detail, stock: direct_manager, managing_director, hr, cutting_manager, stitching_manager, lining_manager, security, store_manager

LOT_WRITERS — can create and correct lots: direct_manager, managing_director, cutting_manager, lining_manager

DM_ONLY — can adjust stock, retire lots, receive, and place/edit orders: direct_manager, managing_director

A role outside the group gets **403** before any body validation happens.

Error shape

Two different 422 shapes exist — handle both.

Business rejections (raised by the service) come back as a plain string:

```
{ "detail": "LEATHER lot requires: dcm, thickness. (supplied: none)" }
```

Pydantic rejections (wrong type, missing required field) come back as an array:

```
{ "detail": [ { "loc": ["body", "article"], "msg": "Field required", "type": "missing" } ] }
```

A safe renderer: `typeof detail === "string" ? detail : detail.map(d => d.msg).join(", ")`.

Status codes used

Code	Meaning on this service
200	OK
201	Lot created / order created
403	Role not permitted for this route
404	Lot, order or supplier id not found
409	A rule was violated — duplicate spec, reserved stock, PO mismatch, wrong order state
422	Field missing, wrong shape, or a value that cannot be accepted

409 is the one to design for. It is never a bug — it is always the backend telling the user "this is a different operation". Every 409 message is written to be shown to the user verbatim.

3. The three rules the UI must respect

Rule 1 — One lot per material spec

A lot's identity is (category, subtype, article, colour, thickness, size). A lot means *what the material is*, not *when it was bought*.

Buying the same article/colour/thickness again is a **top-up of the existing lot** via POST /materials/receive, never a second POST /materials/lots. Attempting the second create returns **409**.

This is why the lot picker can promise "filter article + colour + thickness → exactly one row".

Rule 2 — available is derived, never stored

$\text{available} = \text{on_hand} - \Sigma(\text{active reservations})$

on_hand is a ledger. It moves in exactly two ways: receiving adds to it, cutting subtracts from it. There is no endpoint that sets it directly. The correction path is PATCH /materials/lots/{id}/adjust, which records a **movement with a reason**, so the ledger still adds up.

Never build a UI control labelled "edit quantity". Label it "**Adjust stock**" and make the reason field mandatory-looking.

Rule 3 — Retire, never delete

DELETE /materials/lots/{id} deactivates the lot and retires its barcode. The row survives, and every cut event pointing at it survives. Label the button "**Retire lot**". A scan of the old label afterwards returns **410 Gone**, not 404.

4. The per-category field matrix

This is the heart of the module. Every material class has its own required fields, its own quantity field, and its own unit. **Do not hardcode this table** — fetch it from GET /materials/spec — but here it is so you can design the forms.

Category	Subtype	Required attributes	Quantity field	UOM	Filter boxes
LEATHER	(none)	thickness, dcm	dcm	dcm	article, colour, thickness
LINING	PLAIN_LINING	thickness, mtrs	mtrs	mtrs	article, colour, thickness
LINING	RIBS	kg	kg	kg	article, colour
LINING	KNIT	pcs	pcs	pcs	article, colour
ACCESSORY	BUTTON	size, count	count	pcs	article, colour, size
ACCESSORY	ZIP	size, count	count	pcs	article, colour, size
ACCESSORY	THREAD	thickness, mtrs	mtrs	mtrs	article, colour, thickness
ACCESSORY	OTHER	description, count	count	pcs	article, colour

Three things to note

- **article and colour are required for every material** and are **top-level fields**, not attributes. The required list above covers only the attribute keys *beyond* those two.
- **There is no generic qty field.** The quantity is read from the category's own key. A leather lot sends dcm; a rib lining lot sends kg. Sending qty gets you a 422 for a missing required attribute.
- **LEATHER ignores subtype. LINING with no subtype defaults to PLAIN_LINING. ACCESSORY with no subtype is a 422** — there is no generic accessory quantity.

5. Endpoint reference

5.1 GET /materials/spec — the form definition

Roles: STOCK_READERS **Purpose:** Ask the backend which boxes to render before rendering them. Call this the moment the user picks a category/subtype.

Query params

Param	Required	Notes
category	yes	LEATHER LINING ACCESSORY
subtype	no	PLAIN_LINING/RIBS/KNIT, or BUTTON/ZIP/THREAD/OTHER

Request

GET /api/v1/materials/spec?category=LEATHER

Response 200

```
{
  "category": "LEATHER",
  "subtype": null,
  "filters": ["article", "colour", "thickness"],
  "required_to_add": ["dcm", "thickness"],
  "quantity_field": "dcm",
  "uom": "dcm"
}
```

How to use each field

- filters → which search boxes to show on the stock screen.
- required_to_add → which attribute inputs to render on the Add form, all mandatory.
- quantity_field → which of those inputs is the quantity; give it the numeric keyboard and put the unit beside it.
- uom → the unit suffix to print everywhere.

An unknown combination returns filters: ["article","colour"] and empty required_to_add rather than an error — treat empty required_to_add as "this combination is not configured; block the submit".

5.2 POST /materials/lots — add new material

Roles: LOT_WRITERS **Effect:** Creates the lot, mints its child barcode, and adds the quantity to stock — one transaction, all or nothing.

Request body

```
{  
  "category": "LEATHER",  
  "subtype": null,  
  "article": "SUEDE-A32",  
  "colour": "WHISKY",  
  "attributes": {  
    "thickness": "1.2mm",  
    "dcm": 400  
  },  
  "supplier_id": "3f0c..."  
}
```

attributes is a free-form object whose keys must cover required_to_add exactly. supplier_id is optional.

Response 201

```
{  
  "lot_id": "9a1e...",  
  "lot_barcode": "L-004821",  
  "category": "LEATHER",  
  "subtype": null,  
  "article": "SUEDE-A32",  
  "colour": "WHISKY",  
  "on_hand": 400.0,  
  "reserved": 0.0,  
  "available": 400.0,  
  "uom": "dcm"
```

}

The lot_barcode is the whole point of the 201. Immediately after a successful create, route the user to a print confirmation showing that code — this is the only moment the label gets printed as part of the natural flow. (It can be reprinted later from GET /barcode/materials.)

Errors

Code	When	Message you can show
422	Unknown category	Unknown category 'FABRIC'.
422	Accessory with no subtype	Accessory needs a subtype: BUTTON, ZIP, THREAD or OTHER.
422	Missing article or colour	article is required.
422	Missing attributes	LEATHER lot requires: dcm, thickness. (supplied: thickness)
422	Quantity not a number, or ≤ 0	dcm (quantity) must be > 0 .
409	A lot with this exact spec already exists	Full message includes the existing lot's on-hand and its lot_id

The 409 message reads:

"A lot for SUEDE-A32 · WHISKY · 1.2mm already exists (400.0 dcm on hand). Material is one lot per spec — add this delivery to it with POST /materials/receive using lot_id=9a1e..., rather than creating a second lot."

Design this 409 as a redirect, not an error. Show a dialog: *"This material already exists with 400 dcm on hand. Add this delivery to it?"* with a primary button that opens the Receive form pre-filled. See §7 for the caveat on extracting the id.

5.3 GET /materials/lots — the lot picker

Roles: STOCK_READERS **Purpose:** This is the **only** endpoint that hands you a lot_id. /spec returns a form definition; /stock returns totals with the ids summed away. If a screen needs to name a specific lot, it calls this.

Query params — all optional

Param	Notes
category, subtype, article, colour, thickness, size	Filters. Exact match, case-sensitive except category/subtype which are upper-cased server-side.
sku_id	UUID. Flags the lot this SKU was last cut from.
required	Number. The batch's total need; each lot reports whether it covers it.

Request

GET /api/v1/materials/lots?category=LEATHER&sku_id=7c2b...&required=1200

Response 200

```
{
  "count": 2,
  "lots": [
    {
      "lot_id": "9a1e...",
      "barcode": "L-004821",
      "category": "LEATHER",
      "subtype": null,
      "article": "SUEDE-A32",
      "colour": "WHISKY",
      "thickness": "1.2mm",
      "size": null,
      "uom": "dcm",
      "on_hand": 1500.0,
      "reserved": 100.0,
      "available": 1400.0,
```



```
"last_used_for_sku": true,
"covers_required": true
},
{
  "lot_id": "44b0...",
  "barcode": "L-004822",
  "article": "NAP-11",
  "colour": "FOREST",
  "thickness": "0.9mm",
  "size": null,
  "uom": "dcm",
  "on_hand": 300.0,
  "reserved": 0.0,
  "available": 300.0,
  "last_used_for_sku": false,
  "covers_required": false
},
"options": {
  "article": ["NAP-11", "SUEDE-A32"],
  "colour": ["FOREST", "WHISKY"],
  "thickness": ["0.9mm", "1.2mm"],
  "size": []
},
"suggested_lot_id": "9a1e...",
"required": 1200.0
```

```
}
```

Field-by-field UI meaning

- **options** — the distinct values present across the matched lots. `/materials/spec` says *which* dropdowns to render; `options` says *what goes in them*. Rebuild the dropdowns from `options` on every response so the cascade narrows naturally. An empty array means that dropdown has nothing to offer — hide it.
- **last_used_for_sku / suggested_lot_id** — the lot this SKU was most recently cut from, derived live from production events (not a saved preference, so it never goes stale). Pre-select it, **but show a "last used for this SKU" badge**. The backend's stance is: auto-fill, never silently.
- **covers_required** — null when you didn't pass required. false means this lot's available cannot cover the batch. Grey the row out *before* the cut instead of warning after it.
- **available: 0 rows are returned, not hidden**. A manager searching for a lot they know exists must find it, with the zero explaining itself. Do not filter these client-side.
- **Retired lots are excluded** by the backend. They only appear in `GET /barcode/materials`.

Ordering: suggested lot first, then oldest-created first (FIFO). Preserve the array order.

No pagination. The response returns every matching active lot. Filter hard on the client's first call (always send at least category).

5.4 GET /materials/lots/{lot_id} — one lot

Roles: STOCK_READERS

Response 200

```
{  
  "lot_id": "9a1e...",  
  "barcode": "L-004821",  
  "category": "LEATHER",  
  "subtype": null,
```

```

"article": "SUEDE-A32",
"colour": "WHISKY",
"thickness": "1.2mm",
"size": null,
"uom": "dcm",
"on_hand": 1500.0,
"reserved": 100.0,
"available": 1400.0,
"attributes": { "thickness": "1.2mm", "dcm": 400 },
"supplier_id": "3f0c...",
"is_active": true,
"editable_fields": ["article", "colour", "supplier_id", "thickness"],
"required_attributes": ["dcm", "thickness"]
}

```

editable_fields is the edit form's field list — render exactly those inputs and no others. It is computed per material class, so a leather lot offers thickness and a button lot offers size. You do not need a second call to /materials/spec to build the edit form.

404 if the id is unknown. Note this endpoint **does** return retired lots (is_active: false) — render them read-only with a "Retired" banner.

5.5 PATCH /materials/lots/{lot_id} — correct a lot's identity

Roles: LOT_WRITERS **Purpose:** Fix a typo in an article code, a wrong colour, a mis-entered thickness.

Request body — send only what changed

```
{ "article": "SUEDE-A32", "thickness": "1.4mm" }
```

Accepted keys: article, colour, thickness, size, supplier_id. Nothing else.

Response 200 — the full LotDetail object from §5.4.

What is deliberately not patchable, and why

Field	Why not	Do this instead
category / subtype	They decide the UOM and the whole required-field set. Moving LEATHER→LINING leaves a lot measured in dcm claiming to be metres, and every cut event already pointing at it silently re-denominates.	Retire this lot, create the right one.
on_hand / uom	on_hand is a ledger, not a field. Setting it directly makes stock and history disagree with no record of who changed it.	PATCH /lots/{id}/adjust — same change, with a reason.

Sending any of them returns **422** with a message explaining the alternative. Simply don't render those inputs.

Errors

- **422** — nothing to update (empty body after filtering), or a blocked field was sent.
- **409** — the new spec collides with another lot. Message names the other lot's lot_id and on-hand and tells the user to merge via receive.

5.6 PATCH /materials/lots/{lot_id}/adjust — counted stock correction

Roles: DM_ONLY

Request body

```
{ "delta": -40, "reason": "Physical count 20 Aug — 40 dcm unaccounted" }
```

delta is the movement, not the new total. +12 adds twelve; -12 removes twelve. This is the single most common UI mistake on this endpoint — label the input **"Adjust by"** with an explicit sign, or offer two buttons (Add / Remove) and set the sign yourself.

reason must be 3–300 characters. It is the only record of why the counted stock and the system disagreed, and it lands on an audit row.

Response 200 — the full LotDetail with the new numbers.

Errors

Code	When
404	Lot not found
422	Reason under 3 chars; delta not a number; delta is zero
422	The result would be negative — <i>"That would take SUEDE-A32 to -40.0 dcm. Stock cannot go negative."</i>
409	The result would fall below what is reserved — <i>"100.0 dcm of SUEDE-A32 is reserved for a cut... Release the reservation first."</i>

Show available prominently on this form so the user can see the floor they are adjusting against.

5.7 DELETE /materials/lots/{lot_id} — retire a lot

Roles: DM_ONLY **Returns 200 with a body** — not 204.

```
{
  "lot_id": "9a1e...",
  "is_active": false,
  "barcode_retired": true,
  "on_hand": 60.0,
  "history_preserved": true,
  "message": "SUEDE-A32 is retired. Its barcode no longer scans (410 Gone) and it is hidden from the lot picker; every cut recorded against it is untouched."
}
```

Show message in the success toast — it is written for the user and it answers the question they are about to ask ("did I just delete our history?").

409 while any stock is reserved. **404** if unknown.

Confirmation copy should say **"Retire"**, and should mention that the barcode stops scanning while the cut history stays. Do not use the word "delete" anywhere on this control.

5.8 GET /materials/stock — the stock check

Roles: STOCK_READERS **Purpose:** The DM's aggregate check. Returns **totals across every matching lot**, plus a shortfall and a supplier suggestion.

Query params: category, subtype, article, colour, thickness, size, required — all optional.

Request

GET /api/v1/materials/stock?category=LEATHER&article=SUEDE-A32&colour=WHISKY&required=2000

Response 200

```
{
  "category": "LEATHER",
  "subtype": null,
  "article": "SUEDE-A32",
  "colour": "WHISKY",
  "thickness": null,
  "size": null,
  "uom": "dcm",
  "on_hand": 1500.0,
  "reserved": 100.0,
  "available": 1400.0,
  "lot_count": 1,
  "required": 2000.0,
  "short_by": 600.0,
  "suggested_supplier": { "id": "3f0c...", "name": "Anwar Leathers" }
}
```

Conditional fields — read carefully

- required and short_by appear **only if you passed required**. Otherwise they are absent/null.
- short_by is $\max(0, \text{required} - \text{available})$ — it is 0.0, not negative, when stock covers the need.
- **suggested_supplier appears only when all three hold: required was passed, short_by > 0, and article was passed.** It can still be null if no supplier lists that article. Never assume it is present.
- uom falls back to the category default when lot_count is 0, and to "unit" if no category was given either.

The intended flow this drives: DM filters → CHECK → sees the three numbers → types the required quantity → if short, the ORDER button lights up, pre-filled with article, qty = short_by, and supplier_id from the suggestion.

5.9 POST /materials/receive — approved / rejected receiving

Roles: DM_ONLY **This is also the top-up path** — the correct response to "we bought more of the same material".

Request body

```
{
  "lot_id": "9a1e...",
  "supplier_order_id": "b7d2...",
  "approved_qty": 380,
  "rejected_qty": 20,
  "reserve_for_required": 1200,
  "approve_mismatch": false
}
```

Field	Required	Meaning
lot_id	yes	The lot being topped up

Field	Required	Meaning
supplier_order_id	no	If given, the delivery is matched against the PO and the order flips to ARRIVED
approved_qty	yes, ≥ 0	Added to on_hand
rejected_qty	no, ≥ 0 , default 0	Logged only — never touches stock. This is the supplier quality history.
reserve_for_required	no	Creates an active reservation for this quantity in the same transaction
approve_mismatch	no, default false	DM/MD override — see below

Response 200

```
{
  "lot_id": "9a1e...",
  "on_hand": 1880.0,
  "reserved": 1300.0,
  "available": 580.0,
  "rejected_logged": 20.0,
  "supplier_order_status": "arrived",
  "substituted": false,
  "mismatch_fields": null
}
```

The PO-mismatch flow — the one non-obvious behaviour here

When supplier_order_id is present, the backend compares the delivered lot against the ordered spec on **article, colour, thickness, dcm**. A field the order left blank is **not** a mismatch — ordering "400 dcm of SUEDE-A32" with no colour named means any colour is acceptable.

On mismatch, default behaviour is 409 and nothing enters stock:

"Received material does not match the order on: colour, thickness. A DM/MD may accept it as a substitution with approve_mismatch=true."

Design this as a two-step confirmation. Catch the 409, parse the mismatched field names out of the message, show the user *"This delivery differs from the order on colour and thickness. Accept as a substitution?"*, and on confirm re-POST the identical body with approve_mismatch: true.

⚠ Critical on the substitution path: when the override succeeds, the backend **creates a brand-new lot** for the delivered material and receives into that. The returned lot_id **is not the one you sent**, substituted is true, and mismatch_fields lists what differed. The originally targeted lot is never contaminated with the wrong material.

Your UI must therefore **re-read from the returned lot_id**, not the one it posted, and show a clear "Received into a new substitute lot" notice. Any cached state keyed on the old id is stale.

5.10 POST /suppliers/orders — raise a supplier order

Roles: DM_ONLY **Creates the order in status ordered.**

Request body

```
{  
  "category": "LEATHER",  
  "subtype": null,  
  "article": "SUEDE-A32",  
  "colour": "WHISKY",  
  "thickness": "1.2mm",  
  "dcm": 400,  
  "qty": 600,  
  "supplier_id": "3f0c..."  
}
```

qty must be > 0. colour, thickness, dcm, subtype and supplier_id are optional. The uom is derived server-side from category + subtype — do not send it.

Leave a field blank deliberately when the spec is genuinely open. A field you omit is not checked at receiving; a field you fill *is*. Filling colour "just because you have it" turns a perfectly good delivery of a different colour into a 409 later.

Response 201

```
{  
  "order_id": "b7d2...",  
  "status": "ordered",  
  "article": "SUEDE-A32",  
  "qty": 600.0,  
  "uom": "dcm",  
  "supplier": { "id": "3f0c...", "name": "Anwar Leathers" }  
}
```

Supplier handling — two different behaviours

- **You send `supplier_id`:** the supplier must actually carry that article, else **422** — *"Supplier 'X' does not supply article 'Y'. Choose a supplier that carries it."* And **404** if the supplier id is unknown.
- **You omit `supplier_id`:** the backend suggests one from the article on a best-effort basis. If none matches, the order is still created with `supplier: null` — the DM assigns one later. **No error.** Your UI must render an order with no supplier gracefully.

5.11 PATCH `/suppliers/orders/{order_id}` — mark arrived

Roles: DM_ONLY **Idempotent** — calling it on an already-arrived order is a safe no-op.

Request

```
{ "status": "arrived" }
```

The only accepted value is arrived or ARRIVED (regex-enforced; anything else is a Pydantic 422).

Response 200

```
{ "order_id": "b7d2...", "status": "arrived", "arrived_at": "2026-08-18T09:14:02+00:00" }
```

This is what cues the receiving screen. Note POST /materials/receive with a supplier_order_id flips the status too — so in the normal flow you often don't need this call at all. It exists for "the goods are at the gate but nobody has counted them yet".

5.12 PATCH /suppliers/orders/{order_id}/spec — correct an order

Roles: DM_ONLY Only while the order is still ordered.

Request body — send only what changes

```
{ "article": "SUEDE-A34", "colour": "TAN", "thickness": "1.4mm", "dcm": 420, "qty": 700 }
```

Response 200

```
{ "order_id": "b7d2...", "status": "ordered", "arrived_at": null }
```

409 once the order is arrived — *"Only an ORDERED order's spec may be edited."* So hide the Edit control on any arrived row. Every edit writes a before/after audit row.

5.13 Related barcode endpoints you will need

These live in the barcode module but belong to your material screens.

GET /barcode/materials?category=LEATHER&active_only=true — the material-barcode reprint screen. Roles: MD, DM, HR, Supervisor, Cutting/Lining/Stitching manager.

```
[  
  {  
    "code": "L-004821",  
    "type": "LEATHER_LOT",  
    "status": "active",  
    "caption": "SUEDE-A32 · WHISKY · 1.2mm · 400 dcm",  
    "lot_id": "9a1e...",  
    "category": "LEATHER", "subtype": null,  
    "article": "SUEDE-A32", "colour": "WHISKY",  
    "thickness": "1.2mm", "size": null,  
  }  
]
```

```
"uom": "dcm", "on_hand": 1500.0,  
"label_line": "SUEDE-A32 · WHISKY · 1.2mm · 1500.0 dcm"  
}  
]
```

Encode code as Code128 and typeset label_line underneath — same convention as the piece label. Pass active_only=false to include retired lots; render status: "retired" rows greyed and non-printable.

(Naming note from the backend: the drawer screen's label currently reads "bucket barcode" and should read "drawer barcode". The API has only ever said DRAWER — that rename is frontend-only.)

GET /barcode/resolve?code=L-004821 — scan a lot label. Any authenticated user.
Returns the code's type plus a lot payload:

```
{  
  "code": "L-004821",  
  "type": "LEATHER_LOT",  
  "active": true,  
  "caption": "SUEDE-A32 · WHISKY · 1.2mm · 400 dcm",  
  "lot": {  
    "lot_id": "9a1e...", "category": "LEATHER", "subtype": null,  
    "article": "SUEDE-A32", "colour": "WHISKY", "thickness": "1.2mm",  
    "size": null, "uom": "dcm", "on_hand": 1500.0, "available": 1400.0  
  },  
  "next_expected_scan": "PIECE"  
}
```

404 unknown code, **410 Gone** for a retired one — treat 410 as *"this lot was retired"*, never *"invalid barcode"*.

POST /barcode/print — Code128-ready payload for one or many codes.

6. How to design the screens

Screen A — Stock Hub (landing)

The default screen for the whole module. Three regions.

Top: the filter bar, built dynamically. User picks Category → Subtype. On that change, call GET /materials/spec and render **only** the boxes in filters. Leather and thread get a Thickness box; buttons and zips get a Size box; ribs, knit and "other" get just Article and Colour. Never render a static filter set — the whole point of /spec is that the form matches the material class.

Middle: the three numbers. Render on_hand, reserved, available as three tiles with the uom as a suffix. Give available visual primacy — it is the number that decides whether work can start. Put a small info affordance on reserved explaining it is stock committed to a cut.

Bottom: the requirement + shortfall. A "Required qty" input and a CHECK button that recalls /materials/stock with required. If short_by > 0, reveal a shortfall banner and an **ORDER** button that opens Screen E pre-filled with article, qty = short_by, and supplier_id from suggested_supplier. If suggested_supplier is null, open the order form with the supplier field empty and unblocked — an order without a supplier is legal.

Also give this screen a lot list (Screen B) so the user can drill into a specific lot rather than only seeing totals.

Screen B — Lot list & detail

Drive the list from GET /materials/lots with the current filters. Columns: barcode, article, colour, the class-specific field (thickness or size), on-hand, reserved, available, uom.

Show zero-availability rows normally with the available: 0 visible. Do not hide them — a manager searching for a lot they know exists must find it, and the zero is the explanation.

Row click → GET /materials/lots/{id} → detail panel with:

- the identity block,
- the three numbers,
- the barcode with a **Reprint** action,
- three actions gated by role: **Edit** (LOT_WRITERS), **Adjust stock** (DM_ONLY), **Retire** (DM_ONLY).

Render the Edit form from `editable_fields` — do not hardcode it. If `is_active` is false, show a "Retired" banner and disable all three actions.

Screen C — Add New Material

A two-step form, because the field set does not exist until step one is answered.

Step 1 — Class. Category, then Subtype. Accessory must not allow "no subtype"; make it a required select. Lining with no subtype is legal (defaults to plain).

Step 2 — Fields. Fetch `/materials/spec`, then render: Article (required), Colour (required), plus one input per entry in `required_to_add`. Mark the `quantity_field` input as numeric with the uom printed beside it and a "must be greater than zero" hint.

Submit builds attributes from the step-2 inputs — article and colour go at the **top level**, everything else inside attributes.

Two outcomes to design:

- **201** → a print confirmation showing `lot_barcode`. This is the natural moment the label gets printed; don't bury it in a toast.
- **409 duplicate** → a "this material already exists" dialog with the on-hand quantity from the message and a primary button that opens Screen D (Receive). This should feel like a helpful redirect, not a failure.

Screen D — Receiving

Reachable from an arrived supplier order, from the duplicate-lot 409, and directly from a lot's detail panel.

Fields: the target lot (read-only if arrived from a lot), optional supplier order, **Approved qty**, **Rejected qty**, and an optional "Reserve for requirement" input.

Make the approved/rejected split visually explicit. They are different things: approved enters stock, rejected only enters the supplier's quality history. Two separate labelled inputs, never one field with a toggle.

Handle the mismatch flow properly. On 409, do not show a raw error. Show a substitution confirmation naming the differing fields, and on confirm re-POST with `approve_mismatch: true`. Then **re-read from the returned lot_id** — on substitution it is a new lot. Show a distinct "Received into a substitute lot" result state rather than the normal success state.

Screen E — Supplier Order

Fields: category, subtype, article, colour, thickness, dcm, qty, supplier.

Add a hint under the optional spec fields: *"Fields you fill here are checked against the delivery. Leave a field blank if any value is acceptable."* This single sentence prevents most of the 409s at receiving.

Order rows show status as a two-state chip: **Ordered** → **Arrived**. On an ordered row show **Edit spec** and **Mark arrived**; on an arrived row show neither, and instead show **Receive**.

Screen F — Lot picker (embedded in the cut screen)

This is not a material screen — it is a component the production cut screen embeds. It is what lets the frontend supply leather_lot_id / lining_lot_id to POST /production/log.

The flow:

1. GET /materials/lots?category=LEATHER&sku_id=<sku>&required=<dcm per piece × piece count>
2. Fill the cascading dropdowns from options.
3. Pre-select the row where last_used_for_sku is true, **with a visible badge** — never silently.
4. Grey out rows where covers_required is false, so a lot that cannot cover the batch is ruled out before the cut instead of warning after it.
5. Narrow with &article=&colour=&thickness= as the user picks. One lot per spec means a full filter lands on exactly one row.
6. Take that row's lot_id into the production log payload.

Use category=LINING for the lining cut screen — the suggestion is computed against the lining column, matching which cut will consume it.

7. Two gaps to plan around

These are real holes in the current API, not oversights in this document. Both need a small backend addition; until then, work around them as described.

Gap 1 — No reservation-release endpoint

POST /materials/receive can *create* a reservation via reserve_for_required, but nothing releases one. Since both PATCH /lots/{id}/adjust and DELETE /lots/{id} refuse with 409 while stock is reserved, a lot can get stuck: it cannot be adjusted and cannot be retired.

Work around it: treat reserve_for_required as advanced/optional and don't surface it prominently in the receiving form yet. When a 409 cites a reservation, the message is accurate but the user has no self-service fix — surface it as "contact the DM" rather than offering a retry. Ask the backend team for a POST /materials/reservations/{id}/release (or a release-by-lot route) before this ships.

Gap 2 — No supplier list endpoint

There is no GET /suppliers. supplier_id is accepted on POST /materials/lots and POST /suppliers/orders, and suggested_supplier / the order's supplier object come back on reads — but there is no way to enumerate suppliers to populate a dropdown.

Work around it: make the supplier field on both forms optional and driven only by suggested_supplier from the stock check. Don't build a supplier picker yet. Request a GET /suppliers returning {id, name, articles, contact, is_active}.

Minor: the 409 lot id is only in the message text

The duplicate-lot 409 and the spec-collision 409 both name the existing lot's id, but inside the prose of detail, not as a structured field. Rather than regex-parsing the message, re-query GET /materials/lots with the same spec filters — one lot per spec guarantees you get exactly the row you need. That is also more robust if the message wording changes.

Also worth knowing

- **No pagination anywhere** on this service. GET /materials/lots and GET /barcode/materials return everything matching. Always send category at minimum.
- **No GET /suppliers/orders list.** Orders can be created, edited and marked arrived, but not listed. An order-history screen needs a backend addition too.